Basic Code Analysis: NestJS Request Validation Interceptors

Duration : 90 Min.

Scenario

A NestJS application receives payload fields containing non-sanitized HTML strings, causing XSS vulnerability alerts. Write a NestJS Validation Interceptor class to clean fields.

Target File Location & Creation

File to Create/Update: `student_workspace/logging.interceptor.ts` **Input Resource File to Inspect**: `student_workspace/logging.interceptor.ts`

Task Objectives

Perform the following actions inside the `student_workspace` directory: - Implement HTML sanitization logic inside 'sanitize.interceptor.ts'.

Instructions to Perform the Task

1. When your workspace loads in **VS Code**, use the **Explorer** panel on the left to locate your files.
2. Open and inspect `logging.interceptor.ts` inside `student_workspace/`.
3. Open `logging.interceptor.ts` in `student_workspace/` and perform the required modifications.
4. Save your changes (`Ctrl + S` or `Cmd + S`).
5. Open the built-in terminal by clicking **Terminal > New Terminal** from the top menu.
6. Verify your progress by running `python run.py` locally in the terminal.

Validation

Once you have saved your files and verified your progress, return to the platform dashboard and click the **"Run Test" / "Verify"** button. This will automatically evaluate your changes and generate your score!

Grading Criteria

Test Case	Requirement	Marks
TC1	sanitize.interceptor.ts has valid NestJS class	3 Marks
TC2	Implements NestInterceptor intercept() method	3 Marks

TC3	Sanitizes HTML tags from the request body input	3 Marks
TC4	Returns observable mapping response stream	3 Marks
TC5	Regex or library checks implemented	2 Marks
TC6	Handles nested object objects parsing	2 Marks
TC7	No compiler/transpiler syntax errors	2 Marks
TC8	Exports target class correctly	2 Marks

Total Score: 20 Marks

Important Notes

- This is an auto-evaluated question. Ensure all code edits are properly saved and the 'run.py' checks pass before submission.